

TP Kafka



Oui, vous allez devoir y passer ...



Vous utiliserez le message broker Kafka vu précédemment en cours pour ce TP, dont vous trouverez l'image docker ci-dessous.

<https://hub.docker.com/r/apache/kafka>

Vous utiliserez FastAPI pour le back et PostgreSQL pour vos bases de données.

Il est conseillé de conteneuriser chaque brique applicative (producer, consumer, message broker) sous Docker afin de faciliter la mise en place de Kafka. Ce travail pourra être fait en groupe ICC + IA si besoin.

Chaque page du TP est à compléter intégralement avant de poursuivre vers la suivante.

Contexte :

Vous cherchez à développer une application standard permettant à un utilisateur de commander des objets à une boutique. Cette boutique possède elle-même un inventaire, à débiter à chaque commande. Un utilisateur ne peut pas commander un produit qui n'est pas en quantité suffisante dans l'inventaire.

Proposez un exemple d'architecture microservices pour développer cette application, puis mettez le en oeuvre une fois validé.



Créez les 3 microservices suivants :

- Service de commandes
 - Crée les commandes
 - Les enregistre dans sa base de données
 - Publie les événements `order.created` sur Kafka
 - Implémente le modèle Outbox
- Service d'inventaire
 - Base de données locale dédiée
 - Consomme les événements `order.created`
 - Vérifie et met à jour les stocks
 - Publie les événements `inventory.updated`
- Service de notifications
 - Stateless
 - S'abonne aux événements pertinents
 - Envoie des notifications à la console sur l'état d'une commande



Améliorez vos microservices en y ajoutant :

- Service d'inventaire
 - Permet à un user admin de modifier l'inventaire
- Nouveau service de paiement
 - Consomme les événements `order.created`
 - Vérifie les stocks pour savoir si la commande est possible
 - Publie les événements `payment.succeeded` or `payment.failed`
- Service de notifications
 - Envoie des notifications sur les requêtes admin de l'inventaire et sur l'état des paiements
- Saga chorégraphie : quand un événement **`order.created`** est publié, le service d'inventaire essaie de réserver la commande, et si possible, il renvoie **`inventory.reserved`**; puis le service de paiement charge l'utilisateur et publie **`payment.succeeded`**; si le paiement échoue, le service d'inventaire lit le **`payment.failed`** et libère la commande.

